Design Patterns

An introduction

Michael Mrissa

FAMNIT, Univerza na Primorskem
michael.mrissa@upr.si

October 26, 2018

Reminder: Good Programming Practice

Before using design patterns ...

We must already have "good" programming practice

- Using models like Unified Modeling Language (UML)
  - object-oriented abstract representation
  - Class diagrams (objects)
  - Sequence diagrams (interactions between objects)
- Optimization (Refactoring)
  - Knowing how to identify common points in the code
  - An example: the connection to the database
  - Code separation by function block
- Test your code
  - See the framework `Simpletest` for PHP

Design Pattern: introduction

- Some general information
  - Introduced in 1977 by Christopher Alexander in the field of architecture
  - Description of recurring problems and associated solutions
  - Idea: Maximize reuse of existing knowledge to help solve problems
- Main features
  - Different from a library, abstract solution (no code)
  - Need to customize for a given problem

Definition: Design pattern

Abstract technique for solving a recurring problem

Notion of Design Pattern: introduction

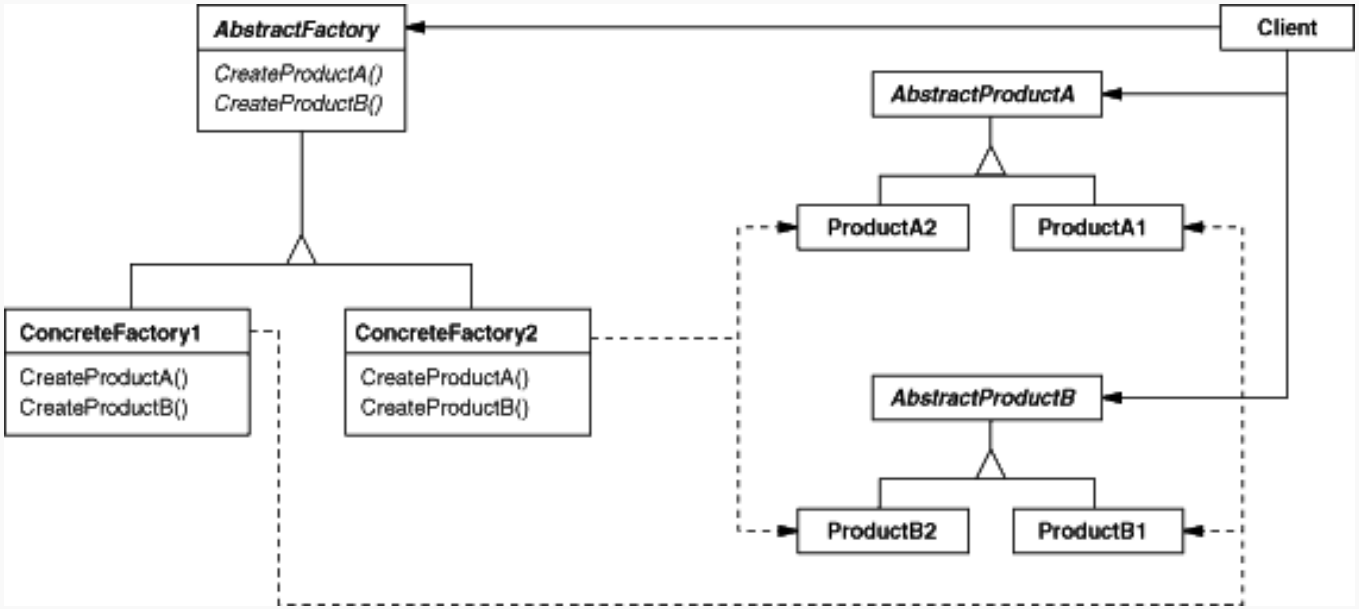
- In three words
  - a **name**: intention of the code in one word
  - a **problem**: application domain
  - a **solution**: implementation, advantages / disadvantages
- Inversion of control
- 3 Design categories patterns [GoF]
  - **Creative**: Abstract Factory, Builder, Factory Method, Prototype, Singleton
  - **Structural**: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
  - **Behavioral**: Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

A basic reference in IT

[GoF] E. Gamma, R. Helm, R. Johnson, J. Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software (1995)

The “Abstract factory” pattern

- Problem
  - We want to create an instance of an interface
  - but when we write the code
    $\Rightarrow$ we do not know what concrete type the instance is going to be
  - Ex: XML documents, window borders …
- Solution
  - Provides an interface for creating objects from the same family
  - Brings independence from the concrete factory implementation
  - The abstract factory will decide what factory we use



The “Factory method” pattern

- Problem
  - Related to the previous pattern
  - Once we have the factory
  - Factory method provides flexibility for choosing the builder implementation.
- Solution
  - Redundant code sharing, implementation of different constructors of concrete classes

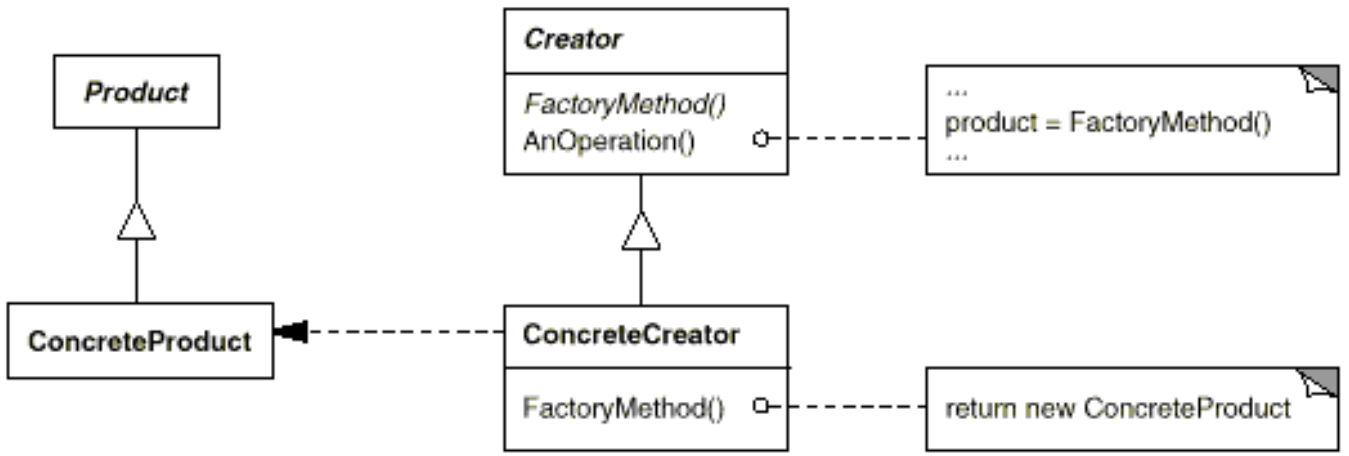


Figure: Factory Method

The “Decorator” pattern

- Problem:
  - Limitations of inheritance
    $\Rightarrow$ Changing behavior after instantiation?
    $\Rightarrow$ Multiple extensions of the same class?
    $\Rightarrow$ Added capacity only to some instances of the class?
- Solution:
  - Create a class that provides an additional feature, such as the decorator
- Benefits
  - Prevents the explosion of subclasses
  - allows you to assign abilities individually
  - dynamically and transparently

The “Decorator” pattern

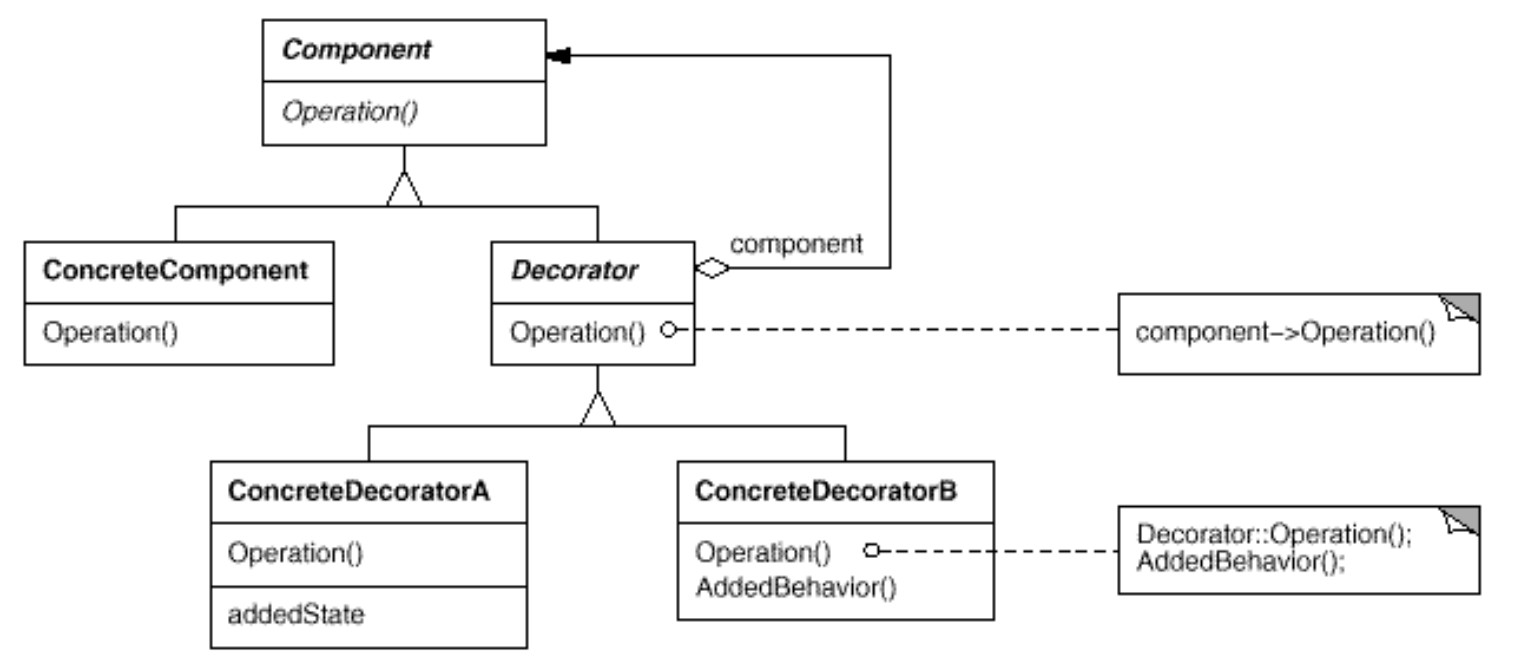


Figure: The decorator design pattern

- Component (VisualComponent): common interface to objects associated with the decorators
- ConcreteComponent (TextView): the object to which the decorator’s capabilities are attached
- Decorator: references the Component object and defines decorator interfaces according to the common interface
- ConcreteDecorator (BorderDecorator, ScrollDecorator): adds capabilities to ConcreteComponents

The “Composite” pattern

- Problem
  - We want to create objects that are made up of several other objects
  - Ex: drawing, or kitchen software
- Solution
  - We define a component object that is an interface for all objects forming the composition
  - We declare a class to manage children (add, remove, etc.)

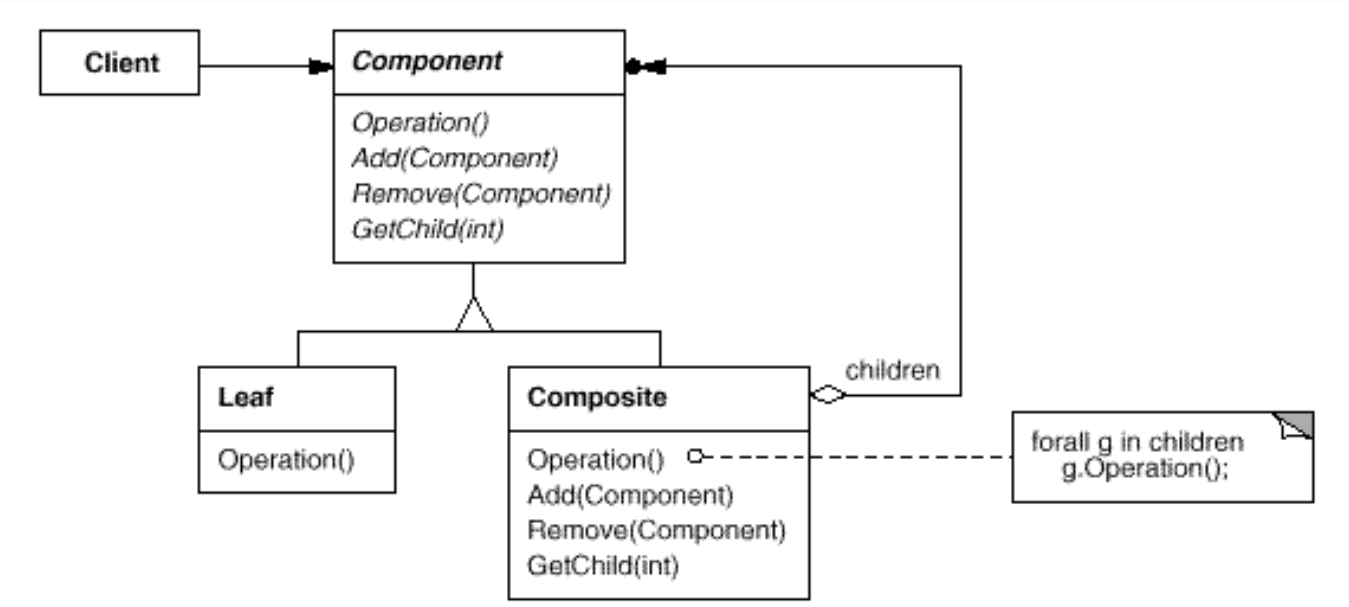


Figure:

The “Strategy” pattern

- Problem
  - Several related classes have different behaviors
  - We hide the organization of classes from outside or we propose a choice
  - Ex: spell (several algos)
- Solution
  - We define a superclass that includes the associated classes

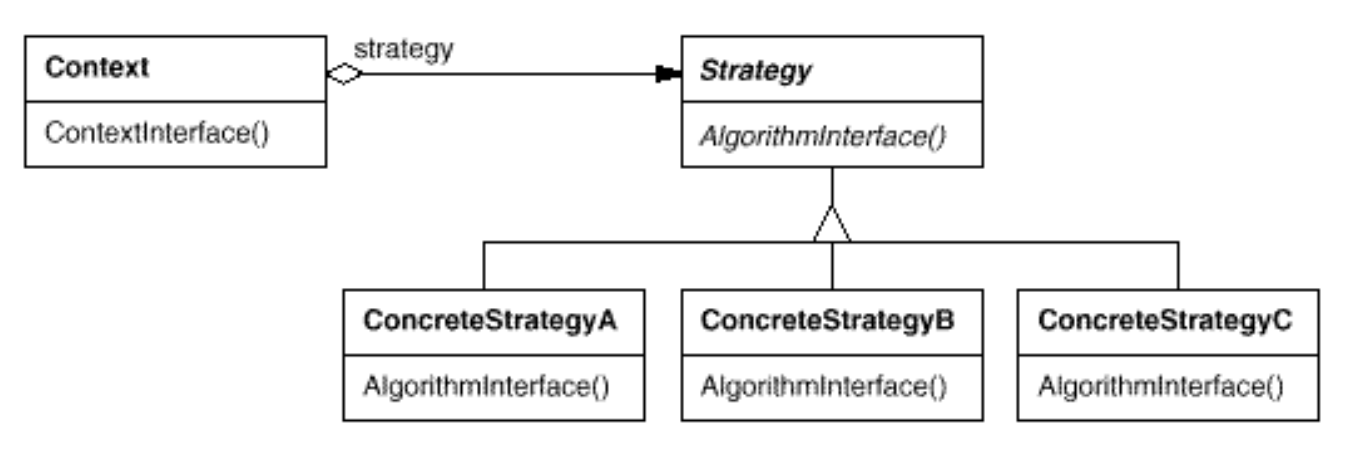


Figure:

Strategy

The “Adapter” pattern

- Problem
  - We want to make several existing classes conform to the same interface
  - It is expensive to redefine their interfaces to fit a common interface
- Solution
  - by inheritance or by composition
  - the adapter passes client requests from the interface to the adapted object
  - implies that the missing functions are implemented to answer the interface

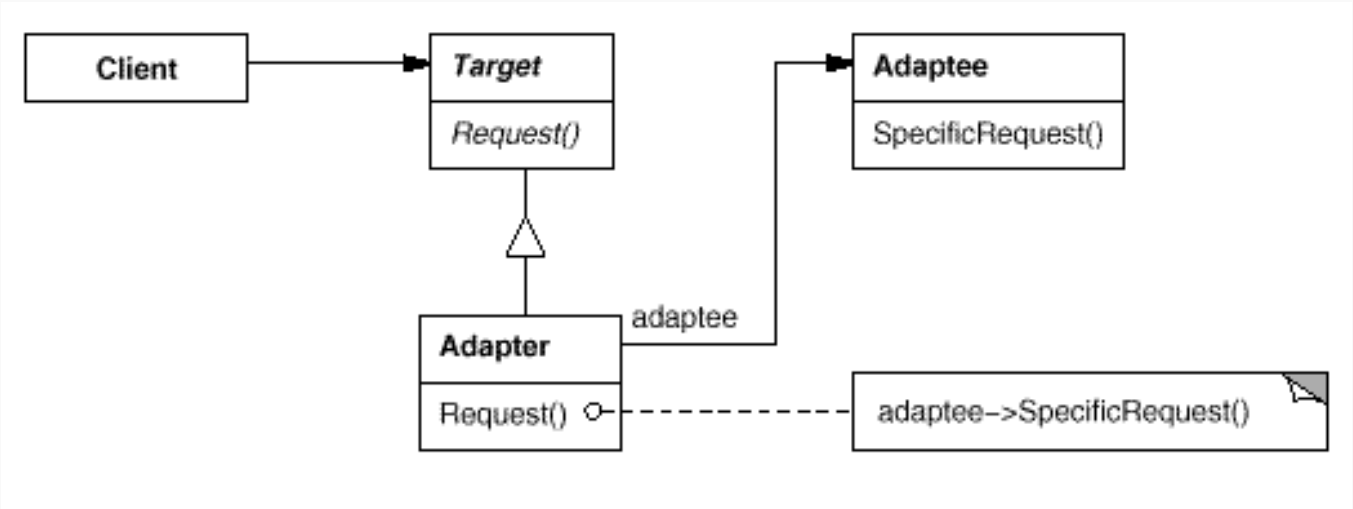


Figure: Adapter (by composition)

The “Proxy” pattern

- Problem
  - We want to reference an object by adding features
  - For example, loading, security (access control), persistence, etc.
- Solution
  - A proxy is used as a copy of the original object
  - Access to the proxy, which performs the operations dynamically according to the client’s requests

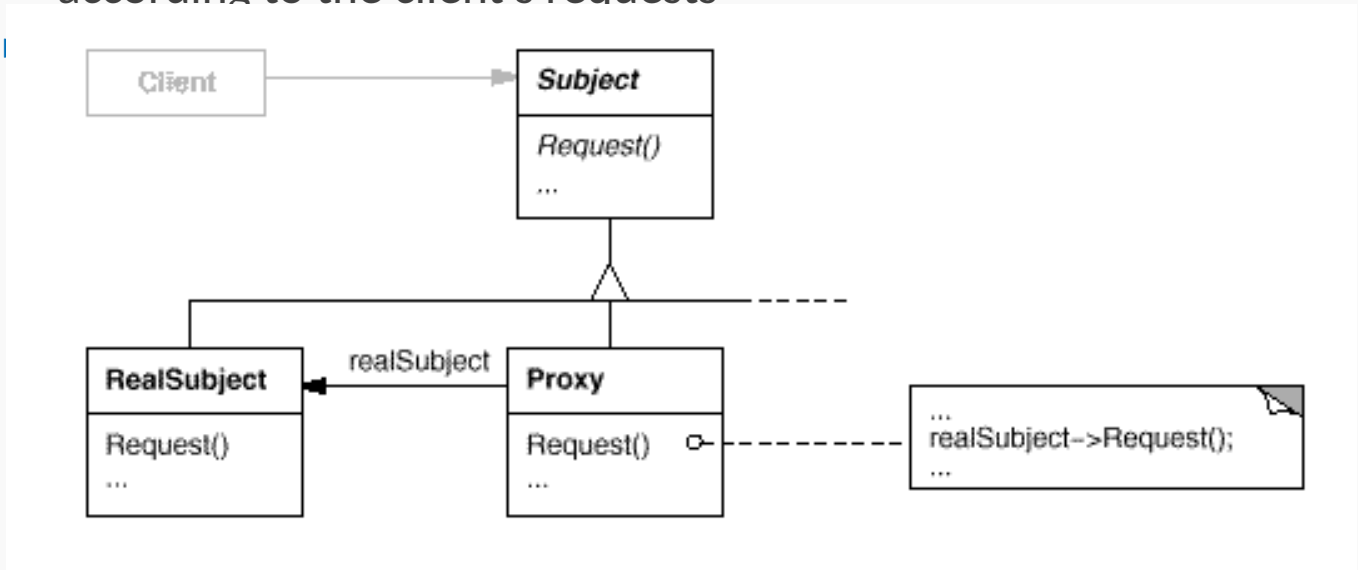


Figure: Proxy

The “Observer” pattern

- Problem ⇒ Identify when an object changes state
- Solution
  - One or more “Observer” classes representing “clients”
  - An “Observable” class representing the monitored object
  - A registration system to be notified of changes

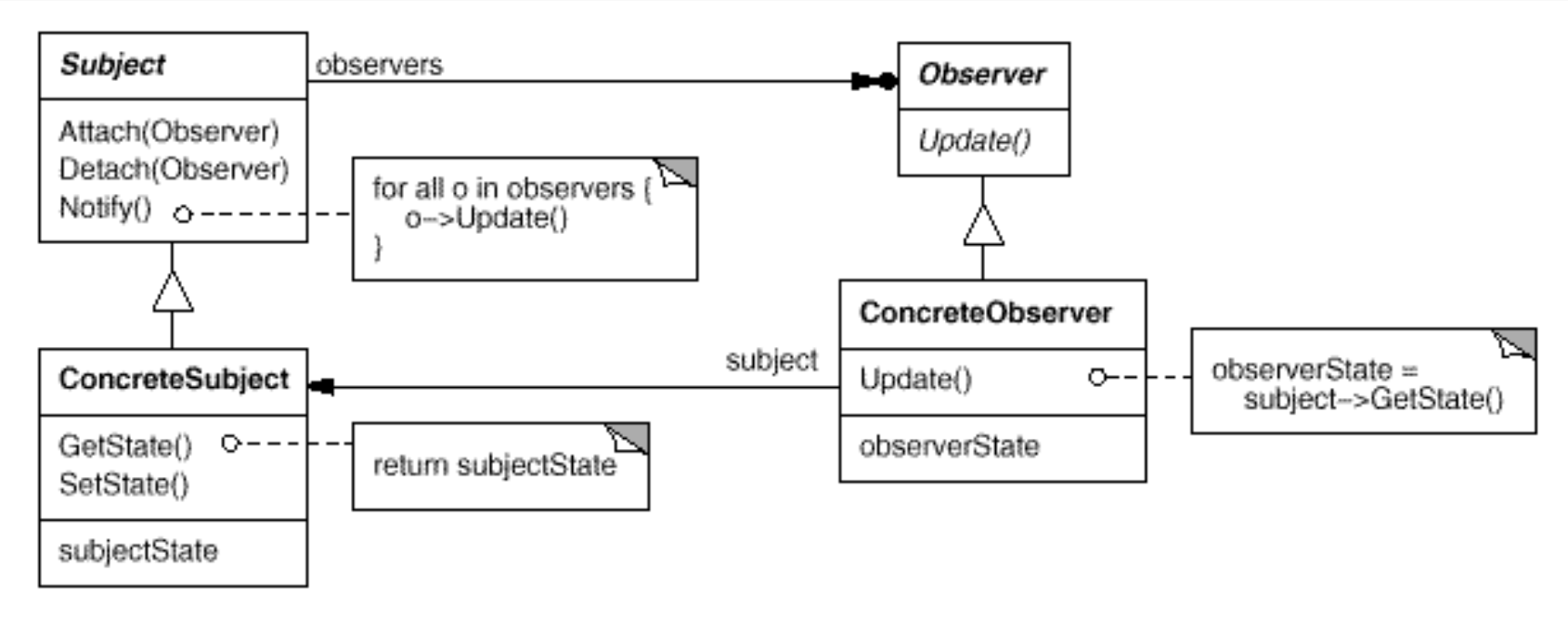


Figure: Observer

The “MVC pattern”

- Problem
  - Separation of concerns for Web architectures
- Solution
  - Model: business logic, controller: routing, view: presentation
  - Applies to the architectural level, in evolution (MV*)

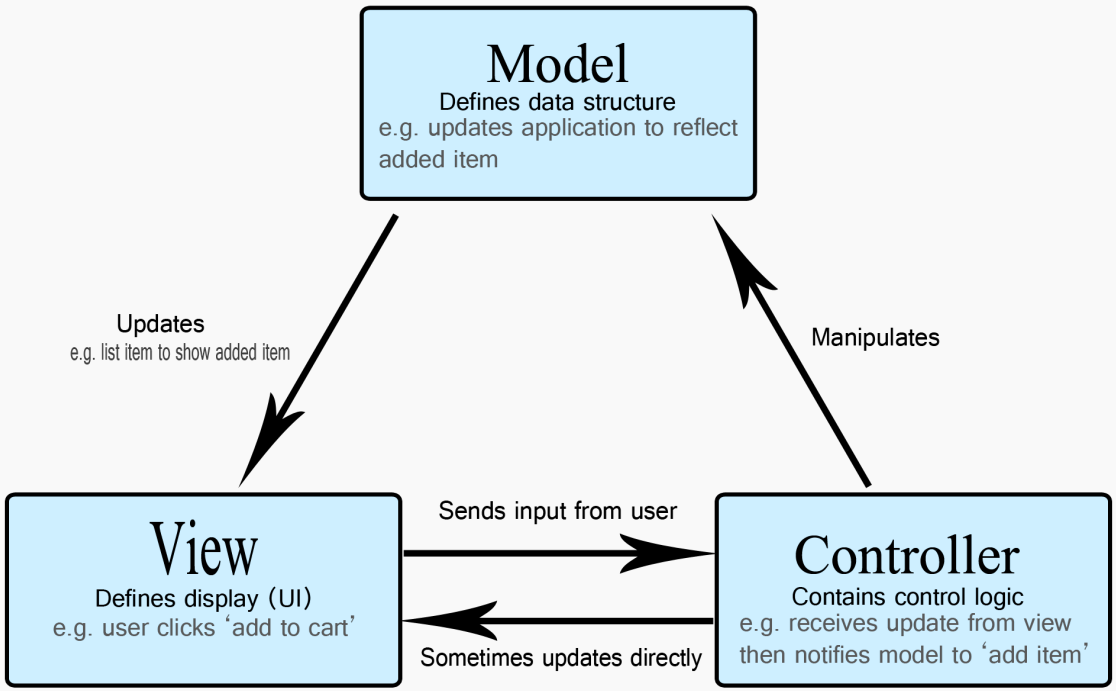


Figure: The MVC design pattern (Source: Mozilla.org)

Conclusion

- Many design patterns
- Mostly object-oriented
  $\Rightarrow$ but totally applicable outside of the object-oriented paradigm
- General consistency
- “Breaking” the limitations of OOP
- Architecture in the code